Integración continua y contenerización en Ruta Estadística

Juan Sebastián Moreno Díaz

Politécnico Grancolombiano · Énfasis Profesional I en Integración Continua

Resumen

Este estudio evalúa Ruta Estadística frente a una rúbrica de integración continua y contenerización de 150 puntos. La demostración reúne una aplicación web en Python y una API .NET que ejecuta R en una red de Docker Compose. GitHub Actions construyó las dos imágenes y verificó el flujo de carga, ejecución y descarga entre contenedores. El motor procesó seis mediciones sintéticas y devolvió una media de 4,16666666666667, dentro de la tolerancia definida para 25/6. El artículo documenta los controles de acceso, el procedimiento reproducible y la evidencia de los ocho criterios.

Palabras clave. integración continua, Docker Compose, comunicación entre servicios, reproducibilidad, R.

1 Introducción

Una aplicación estadística reproducible permite identificar los datos, las opciones y el método que originaron cada resultado. Ruta Estadística guía al usuario desde la pregunta y la importación de datos hasta la interpretación y la exportación. El escritorio puede invocar el paquete R como proceso local o delegar el cálculo a un servicio HTTP configurado para Docker (Moreno Díaz, s. f., `LEEME.md`).

La guía del curso vincula la integración continua con las pruebas automatizadas y la detección temprana de errores. Su primera entrega pide dos contenedores comunicados y las siguientes mencionan Jenkins, Travis CI y Codeship (Politécnico Grancolombiano, s. f., pp. 2–8). La rúbrica aplicada especifica GitHub Actions y exige la construcción de las imágenes.

El estudio compara la arquitectura inicial con la rama de entrega y distingue los informes históricos de la ejecución evaluada. El intercambio entre contenedores concentra 35 de los 150 puntos.

2 Método y alcance

2.1 Fuentes y procedimiento de revisión

La unidad de análisis es la rama entrega/ruta-estadistica-compose del repositorio mljuansmoreno/p005Lab. El README identifica al autor y la institución. La revisión combina archivos de implementación, registros de ejecución y la guía del curso. El ZIP se conservó como referencia de la configuración inicial.

Se revisaron los dos archivos de Compose, sus Dockerfiles, el flujo de Actions y las pruebas HTTP. Los resultados se vincularon al SHA de la ejecución de Linux. El trabajo de Windows se interpretó por separado porque ejecuta R y la API como procesos nativos.

2.2 Estados de la matriz

Verificado. La evidencia disponible respalda todos los elementos aplicables del criterio.

Parcial. Hay componentes o evidencias comprobables y quedan condiciones de la rúbrica por satisfacer.

Sin evidencia suficiente. La entrega carece de registros que permitan comprobar la ejecución o el resultado alegado.

No implementado. Los archivos de la entrega muestran que falta un componente indispensable del criterio.

Los valores de la Tabla 1 representan puntajes máximos. Se conservan los ocho pesos de la rúbrica para que el evaluador pueda rastrear cada hallazgo hasta la evidencia correspondiente.

2.3 Organización del proyecto e historial

La solución RutaEstadistica.sln agrupa el escritorio y las capas .NET. La interfaz de demostración y su prueba se ubican en aplicacion/. El servicio motor conserva su Dockerfile en contenedores/, la API en codigo/ y el paquete de cálculo en motor_r/. La orquestación de la demostración se define en docker-compose.yml, en la raíz.

El README documenta la preparación, compilación, arranque, prueba y apagado. La entrega se publicó como PR en borrador (Moreno Díaz, 2026). El commit asociado a la evidencia de Linux es 3434ff2f5c7dc836636644299a6957eafad16f57.

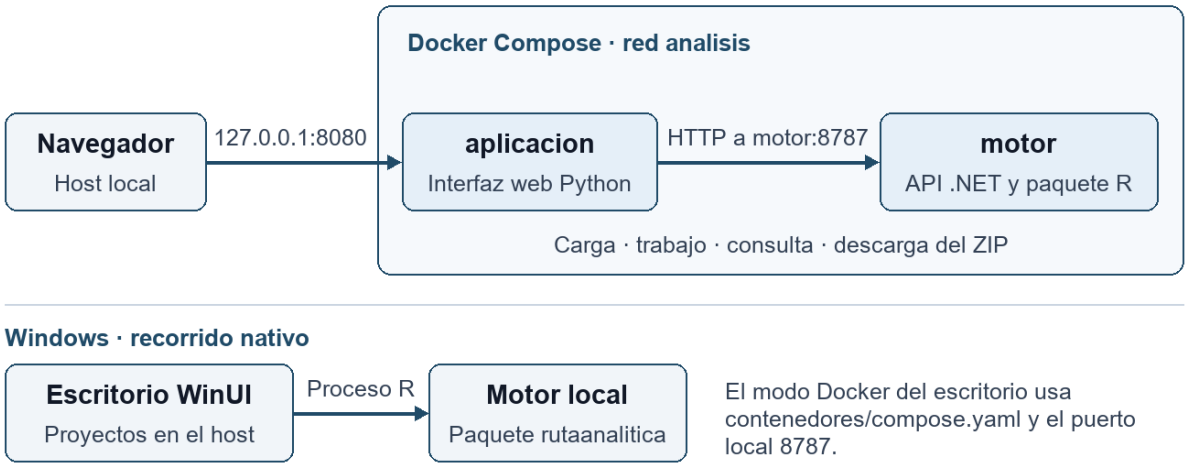
3 Arquitectura y flujo de datos

3.1 Topología implementada

El escritorio WinUI/.NET se ejecuta en Windows. Conserva sus proyectos en el host y puede usar el motor local o el servicio de contenedores/compose.yaml. La demostración de dos contenedores tiene un punto de entrada separado. El navegador accede a la interfaz de Python por loopback y el servidor web solicita el análisis a la API .NET por la red de Compose. Ambas modalidades usan el paquete rutaanalitica.

Figura 1

Arquitectura de la demostración y del escritorio



Nota. Elaboración propia a partir de docker-compose.yml, contenedores/compose.yaml, aplicacion/web.py y documentacion/arquitectura.md.

3.2 Intercambio con la API de trabajos

La API recibe el CSV mediante POST /datos y devuelve un identificador, un recuento y una huella. POST /trabajos admite la solicitud con HTTP 202. La aplicación consulta GET /trabajos/{id} y descarga un ZIP con resultado.json y figura.png. El servidor comprueba la pertenencia del trabajo al proyecto. El resultado incluye los identificadores de solicitud y de proyecto y las huellas de los datos y la preparación.

El proceso que envía el CSV y consulta la API corre dentro de aplicacion. Usa el nombre DNS motor en el puerto 8787. El motor monta su directorio temporal en memoria y el escritorio conserva sus propios datos en Windows.

4 Contenerización y seguridad

4.1 Imágenes de los servicios

La imagen de la aplicación usa la base oficial python:3.13.7-slim-bookworm. Declara que sus dependencias se limitan a la biblioteca estándar, expone 8080 y arranca con python web.py (Docker, s. f.-a). El Dockerfile del motor publica la API autocontenida para linux-x64 con el SDK de .NET 10.0.403 (Microsoft, s. f.) y usa rocker/r-ver:4.5.2 para ejecutar R (Rocker Project, s. f.). El archivo bases.lock.json registra los digests de las dos bases del motor.

La imagen del motor verifica el SHA-256 de jsonlite 2.0.0 antes de instalarlo y añade rutaanalitica. La lista de bibliotecas del sistema se resuelve con apt durante la construcción. La API escucha en 0.0.0.0:8787 dentro del contenedor y comprueba su salud mediante una petición autenticada a /salud.

4.2 Servicios y red

El proyecto ruta-estadistica-demo declara los servicios aplicacion y motor en la red analisis. Solo aplicacion publica un puerto en el host, ligado a 127.0.0.1. El motor recibe el tráfico en 8787 dentro de Docker. Compose espera el estado saludable del motor antes de iniciar la aplicación. La Figura 5 identifica la red efectiva y sus miembros (Docker, s. f.-c).

4.3 Protección de credenciales

La plantilla .env.example contiene una ruta ficticia y un puerto. Git excluye .env y los archivos de credenciales. El contexto de compilación usa una lista de rutas permitidas y excluye los patrones de secretos. Compose monta el archivo externo en /run/secrets/motor y los dos servicios lo leen en ejecución. CI genera una credencial temporal con OpenSSL (Docker, s. f.-b).

La API exige Bearer, rechaza Origin y admite Host motor mediante una opción explícita de Compose. Ambos contenedores ejecutan como usuario sin privilegios y con la raíz de solo lectura. Las cargas en memoria y los trabajos terminados se depuran después de 15 minutos al ingresar nuevas cargas o solicitudes. Los archivos de trabajo residen en un tmpfs que se descarta al detener el contenedor.

5 Integración continua y resultados

5.1 Ejecución de GitHub Actions

El flujo .github/workflows/verificacion.yaml se activa en push y pull_request. El trabajo de Windows restaura dependencias, comprueba el motor nativo y compila la solución. El trabajo Docker Linux construye ambas imágenes, espera la salud de Compose, registra el estado y la red, y ejecuta la prueba desde aplicacion. La etapa final detiene los servicios incluso si falla una etapa anterior (GitHub, s. f.).

El trabajo Docker Linux concluyó correctamente para 3434ff2f5c7dc836636644299a6957eafad16f57. La ejecución y su artefacto de registros permiten contrastar las Figuras 2 a 6. El PR se conserva en borrador.

Registros y artefacto en GitHub Actions, ejecución 42565818761.

5.2 Resultados de las comprobaciones

El informe `pruebas_integracion.txt` registra 28 comprobaciones correctas de una ejecución anterior. `pruebas_integracion_20261030.txt` registra 50 y `adaptador_http.txt` registra 51, incluida una comprobación del adaptador frente a la API real en Windows. Las cifras corresponden a informes distintos y se leen por separado. Las comprobaciones abarcan importación, SQLite, contratos, exportación reproducible y ejecución con el trabajador R (Moreno Díaz, s. f., `documentacion/estado_entrega.md`, `documentacion/continuidad.md` y `documentacion/informes/`).

El cálculo verificado en Linux se originó en aplicacion y llegó a motor:8787. R procesó seis filas y devolvió 4,16666666666667. El identificador 93dc03f0d5b04ed69cfecde447fe88f5 permite localizar la salida exacta de esa prueba en la Figura 6. El flujo auditó la ausencia del secreto montado en las imágenes y de su valor en los registros.

5.3 Límites de la evidencia

La comprobación archivada de MotorDocker se hizo contra un proceso en Windows y respalda la compatibilidad del adaptador HTTP. Las Figuras 2 a 6 proceden del ejecutor Linux de GitHub Actions. La prueba de esta entrega cubre la ruta descriptiva del CSV sintético. Las siete rutas de análisis corresponden a la prueba nativa de Windows.

6 Matriz de evaluación

Tabla 1

Trazabilidad de la rúbrica de integración continua

Criterio	Pts.	Estado	Hallazgo	Fuente o figura
1 · GitHub	15	Verificado	Clon con historial, rama de entrega, PR y comandos reproducibles.	README.md, § 2.3
2 · Aplicación	20	Verificado	Imagen Python oficial, dependencias declaradas, puerto y arranque. Compilación confirmada.	aplicacion/Dockerfile, Fig. 2
3 · Motor	15	Verificado	API .NET y R configurados por entorno con almacenamiento temporal.	Dockerfile del motor, Fig. 2
4 · Compose	20	Verificado	Dos servicios saludables en red propia. Solo la web se publica en loopback.	docker-compose.yml, Figs. 3–5
5 · Comunicación	35	Verificado	Carga y resultado verificados por ID y SHA-256. R devuelve seis filas y media 25/6.	aplicacion/web.py, Fig. 6
6 · Seguridad	15	Verificado	Secreto temporal montado, exclusiones y revisión de imágenes y registros.	.env.example, § 4.3
7 · CI	15	Verificado	GitHub Actions construyó, probó y apagó Compose en Linux.	verificacion.yaml, § 5.1
8 · Documento	15	Verificado	DOCX y PDF con diagrama, referencias y cinco capturas de registros.	Figs. 1–6, § 8
Total	**150**		**Puntaje máximo de la rúbrica.**	

Nota. Los valores corresponden a los máximos de la rúbrica. Las figuras remiten a la ejecución de Linux asociada al SHA indicado en la sección 5.1.

La prueba de ida y vuelta aporta la evidencia funcional para el criterio de 35 puntos. Las salidas de compilación, arranque y red identifican las condiciones en que se ejecutó.

7 Procedimiento y evidencias

7.1 Preparación y construcción de las imágenes

Desde la raíz del repositorio, compruebe con docker info que el daemon usa contenedores Linux y revise docker compose version. Genere un archivo privado fuera del repositorio y configure RUTA_ESTADISTICA_ARCHIVO_SECRETO según el README.

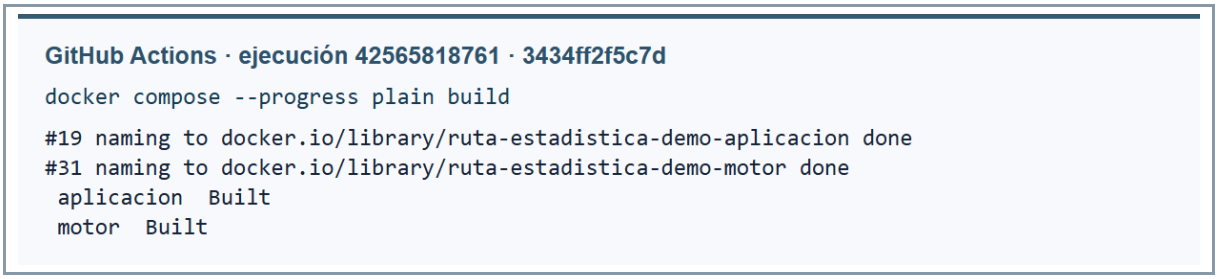
```
docker info --format '{{.OSType}}'
```

Construya las dos imágenes. La Figura 2 registra el paso de compilación en GitHub Actions y las etiquetas de las imágenes resultantes.

```
docker compose --progress plain build
```

Figura 2

Construcción de las dos imágenes



Nota. Extracto del registro de compilación de Docker Compose en el ejecutor Linux.

Compose lee la ubicación del secreto desde el entorno. La ruta es el único valor que debe figurar en la configuración del host.

```
docker compose version
```

La compilación de aplicacion prepara la interfaz y la prueba de extremo a extremo. La del motor publica la API .NET e instala los paquetes R. La salud de los procesos se verifica en el paso de arranque.

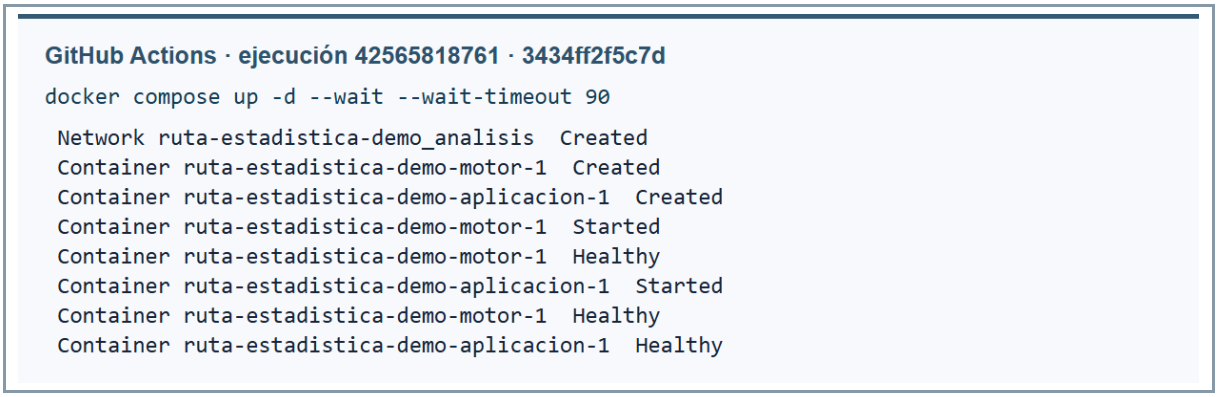
7.2 Arranque y estado de los contenedores

Inicie Compose desde la raíz con el archivo de secreto configurado. La opción --wait espera la salud de ambos servicios.

```
docker compose up -d --wait --wait-timeout 90
```

Figura 3

Arranque del proyecto con docker compose up



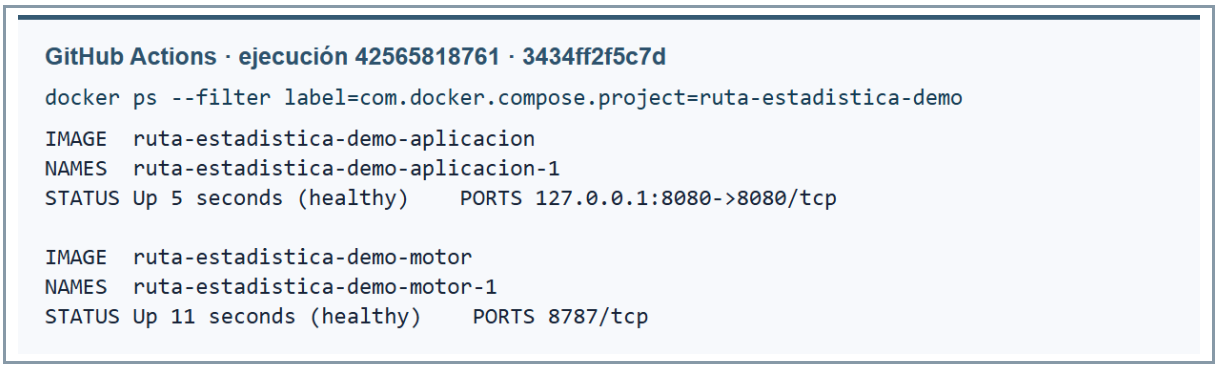
Nota. El motor alcanza el estado saludable y Compose inicia la aplicación.

Liste los contenedores del proyecto y compare las publicaciones de puertos.

```
docker ps --filter label=com.docker.compose.project=ruta-estadistica-demo
```

Figura 4

Estado de los contenedores con docker ps



Nota. Se reorganizaron las columnas del registro para facilitar la lectura. Ambos servicios están saludables y solo la interfaz publica un puerto.

7.3 Inspección de la red

Inspeccione la red declarada en docker-compose.yml. Compose antepone el nombre del proyecto a la clave analisis (Docker, s. f.-c).

```
docker network inspect ruta-estadistica-demo_analisis
```

Figura 5

Inspección de la red con docker network inspect

GitHub Actions · ejecución 42565818761 · 3434ff2f5c7d

```
docker network inspect ruta-estadistica-demo_analisis
```

```
ruta-estadistica-demo_analisis (bridge)
ruta-estadistica-demo-motor-1 172.18.0.2/16
ruta-estadistica-demo-aplicacion-1 172.18.0.3/16
```

Nota. La salida de inspección identifica el controlador bridge y los dos miembros.

Preparación de la prueba funcional

El navegador accede a http://127.0.0.1:8080/ y puede ejecutar el mismo cálculo desde el formulario. La prueba automatizada se ejecuta dentro de aplicacion y llama a esa acción web. El servidor de la aplicación se comunica con la API por HTTP.

Compose configura RUTA_ESTADISTICA_PERMITIR_HOST_MOTOR=1. La API acepta el encabezado Host motor con la credencial correcta. El modo del escritorio conserva la aceptación local de localhost y 127.0.0.1.

7.4 Comunicación de ida y vuelta por nombre de servicio

La columna c001 del CSV compartido contiene 1, 2, 3, 5, 6 y 8. Su suma es 25 y su media es 25/6. Ejecute la prueba de la acción web desde el contenedor aplicacion.

```
docker compose exec -T aplicacion python probar_compose.py
```

La aplicación carga el CSV, encola un descriptivo, consulta el estado y descarga el resultado. La verificación exige el mismo identificador de solicitud y las mismas huellas, seis filas y una media con tolerancia relativa de 10^{-10}.

Figura 6

Prueba de comunicación entre contenedores

GitHub Actions · ejecución 42565818761 · 3434ff2f5c7d

```
docker compose exec -T aplicacion python probar_compose.py
{
  "destino": "motor:8787",
  "trabajo_id": "93dc03f0d5b04ed69cfecde447fe88f5",
  "solicitud_id": "fcf132ad16ee4aa5b0efbedddb0e0bf0",
  "estado": "completado",
  "sha256": "4c4bcaecdea0a6f5169163dbd34fc0c82d8982148e354f92d4431d6114120499",
  "filas_analizadas": 6,
  "media": 4.16666666666667
}
```

Nota. La salida registra el destino interno, la correlación de la solicitud y el valor calculado por R.

7.5 Condiciones de aceptación

El estado completado y la cifra descargada acreditan el intercambio funcional. El hash permite comprobar que el resultado corresponde al CSV sintético compartido con la prueba nativa.

Después de conservar los registros, ejecute docker compose down --remove-orphans y retire el archivo de secreto temporal.

8 Discusión y conclusiones

8.1 Alcance de la reproducibilidad

El uso del mismo paquete R en dos modalidades reduce la duplicación de lógica estadística. Guardar el CSV normalizado, el contrato y el hash permite relacionar una cifra con la revisión de datos que la produjo. En las comprobaciones archivadas, el trabajador R devolvió una media conocida y el guion exportado reprodujo el resultado sobre su propia entrada. Esas observaciones respaldan la trazabilidad del cálculo en el entorno ensayado (Moreno Díaz, s. f., `documentacion/informes/pruebas_integracion_20261030.txt`, líneas 39–49).

La prueba nativa y la prueba de Linux comparten el CSV. Ambas comprueban la media descriptiva con la misma tolerancia relativa. Linux añade la resolución del nombre motor, la comunicación por la red de Compose y el arranque condicionado a la salud. Las dependencias del sistema resueltas por apt todavía pueden variar entre compilaciones.

8.2 Limitaciones de la entrega evaluada

La ejecución Docker se realizó en un runner Ubuntu de GitHub Actions. La demostración utiliza un conjunto sintético y una única ruta estadística. Los informes nativos de Windows cubren otros análisis y la integración del escritorio con el motor. El archivo de trabajo de Linux es temporal y se descarta al detener el servicio.

8.3 Conclusiones

El recorrido entre contenedores completó un análisis descriptivo real. La aplicación descargó del motor un resultado de seis filas con media 4,16666666666667. El identificador del trabajo y las huellas permiten correlacionar la entrada, la solicitud y el resultado.

La compilación y el arranque de las dos imágenes, la inspección de la red y el cálculo se comprobaron en la ejecución de Linux vinculada al commit. El historial, los comandos y los cinco registros permiten repetir la evaluación de los ocho criterios de la rúbrica.

Referencias

Docker. (s. f.-a). *Dockerfile reference*. https://docs.docker.com/reference/dockerfile/

Docker. (s. f.-b). *Manage secrets securely in Docker Compose*. https://docs.docker.com/compose/how-tos/use-secrets/

Docker. (s. f.-c). *Networking in Compose*. https://docs.docker.com/compose/how-tos/networking/

GitHub. (s. f.). *Workflows*. GitHub Docs. https://docs.github.com/en/actions/concepts/workflows-and-actions/workflows

Microsoft. (s. f.). *Official .NET Docker images*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/architecture/microservices/net-core-net-framework-containers/official-net-docker-images

Moreno Díaz, J. S. (s. f.). *Ruta Estadística* [Código fuente, documentación e informes]. Archivo `p005Lab-main.zip`.

Moreno Díaz, J. S. (2026). *Ruta Estadística en Docker Compose* [Solicitud de incorporación de cambios]. GitHub. https://github.com/mljuansmoreno/p005Lab/pull/1

Politécnico Grancolombiano. (s. f.). *Énfasis Profesional I (Integración Continua)* [Guía de proyecto y lecturas del módulo]. Archivo `integracion continua.pdf`.

Rocker Project. (s. f.). *r-ver*. https://rocker-project.org/images/versioned/r-ver

Identificación de la entrega evaluada

El PR en borrador está enlazado en las referencias. El siguiente SHA identifica el commit asociado con los registros de las Figuras 2 a 6.

```
3434ff2f5c7dc836636644299a6957eafad16f57
```